ChargeCheck (차지체크) 앱 아키텍처

충전할 때마다 차가 '실제로 충전됐는지'를 자동 기록하고, 다음 운전자에게 도착 전에 알려주는 Pleos Connect 차량용 앱

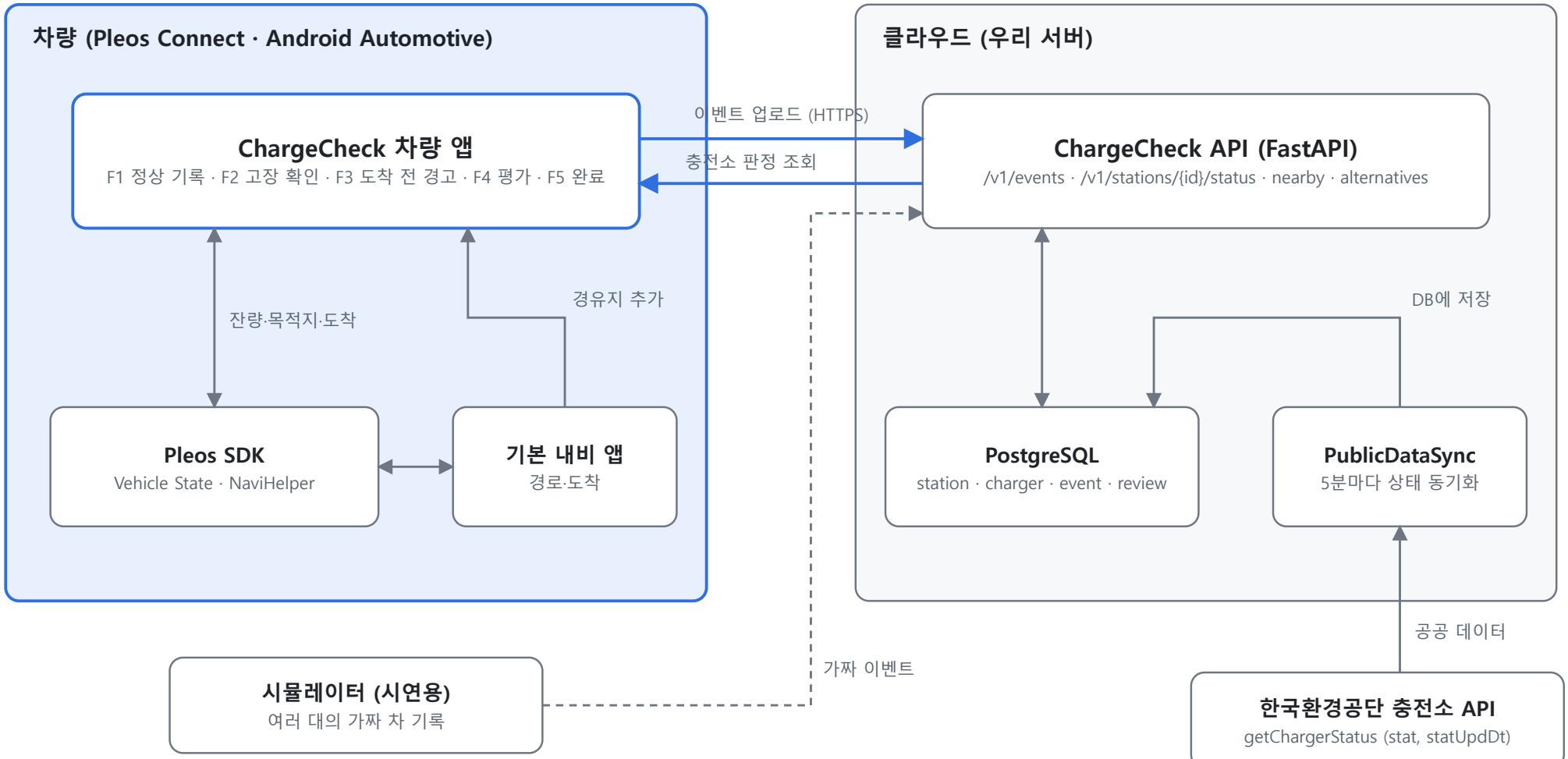
구성 요소	① 차량 앱 **android/** (Kotlin · Compose · Hilt · Room) ② 서버 **server/** (FastAPI · PostgreSQL) ③ 시뮬레이터 **tools/simulator/**
쓰는 Pleos SDK	Vehicle SDK(배터리 잔량 SOC), NaviHelper(목적지·도착·경유지) — 둘 다 지금 신청 없이 사용 가능
외부 데이터	한국환경공단 전기차 충전소 API (getChargerStatus: stat, statUpdDt)
빌드 변형	**real** = Pleos/Android 어댑터 · **demo** = 가짜 어댑터 + 시나리오 재생(시연 영상용)
상세 명세	기획문서/앱구조_명세서_AI용.md (이 PDF와 같은 이름·규칙을 씀. 다른 AI에게는 MD 파일을 주세요)
코드 위치	chargecheck/android (코드 폴더 경로에는 한글을 쓰지 않음)
작성일	2026-10-24

1. 기능 코드는 Pleos SDK를 직접 부르지 않는다. 모든 외부 의존성은 **포트(인터페이스)** 뒤에 둔다.
2. 충전 판단은 **순수 함수 상태 머신**. 부수 효과는 Effect로 돌려준다.
3. 충전 중 차가 꺼질 수 있다 → **LIVE(실시간)**와 **RETRO(도착·출발 비교)** 두 경로.
4. 잔량만으로 고장을 확정하지 않는다. 실패는 **운전자 확인 1회** 후에만 기록. 성공은 자동.
5. 서버에는 좌표를 보내지 않는다. 충전소 ID·결과·SOC·시각만, 기기 ID는 임의 UUID의 해시.

1. 시스템 구성

차량 · 클라우드 · 공공데이터

차량 앱은 충전 결과를 서버에 보내고, 서버는 공공 API 상태와 합쳐 충전소 판정(초록·노랑·빨강·회색)을 돌려준다.

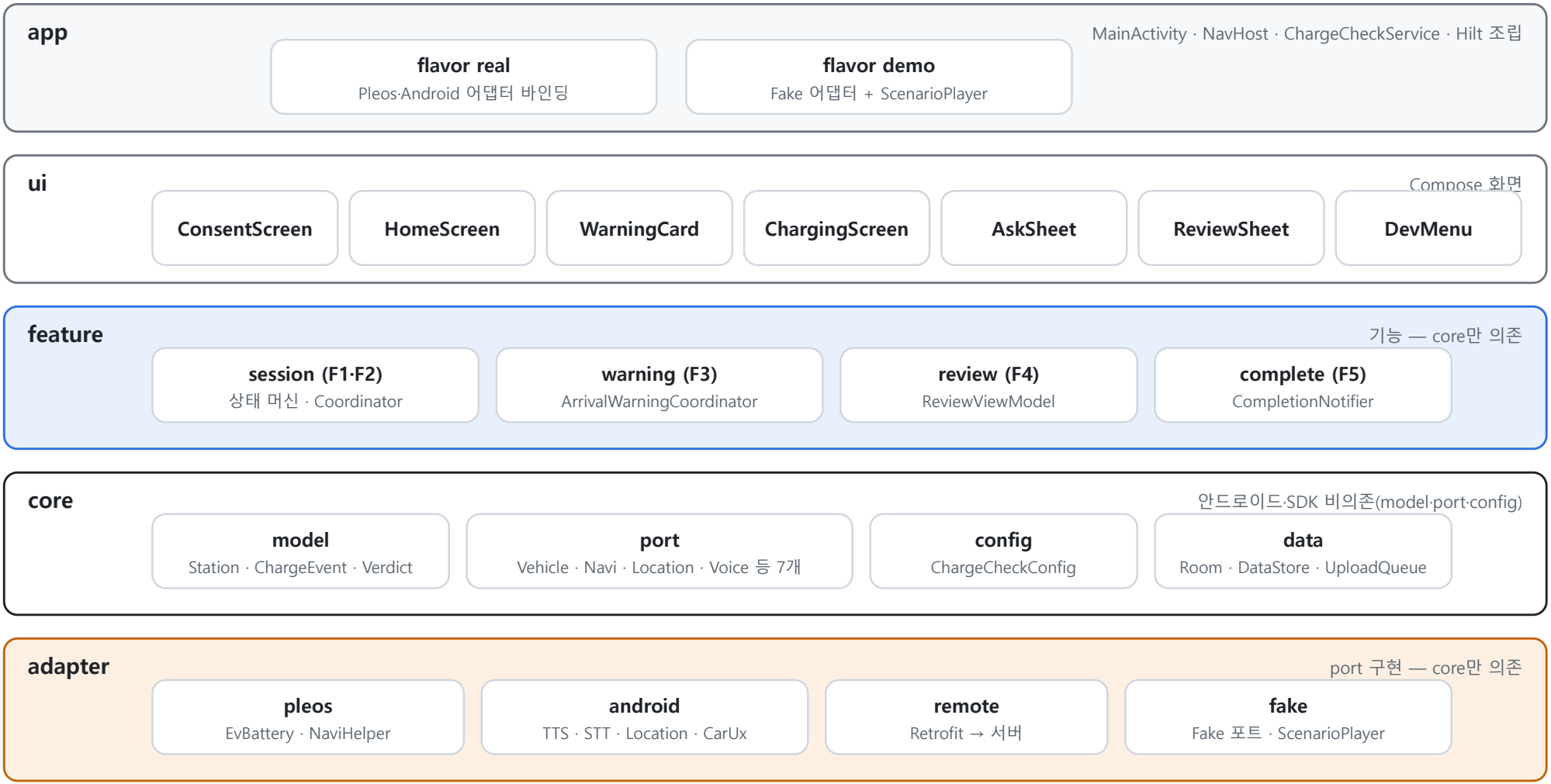


서버로는 충전소 ID · 성공/실패 · SOC · 시각만 보낸다. 좌표와 이동 경로는 보내지 않는다.

2. 차량 앱 모듈 구조

Gradle 멀티 모듈 · 의존은 위에서 아래로만

feature는 core만 안다. adapter도 core만 안다. 둘을 잇는 것은 app(DI 조립)뿐이라, SDK가 바뀌어도 adapter 하나만 고친다.

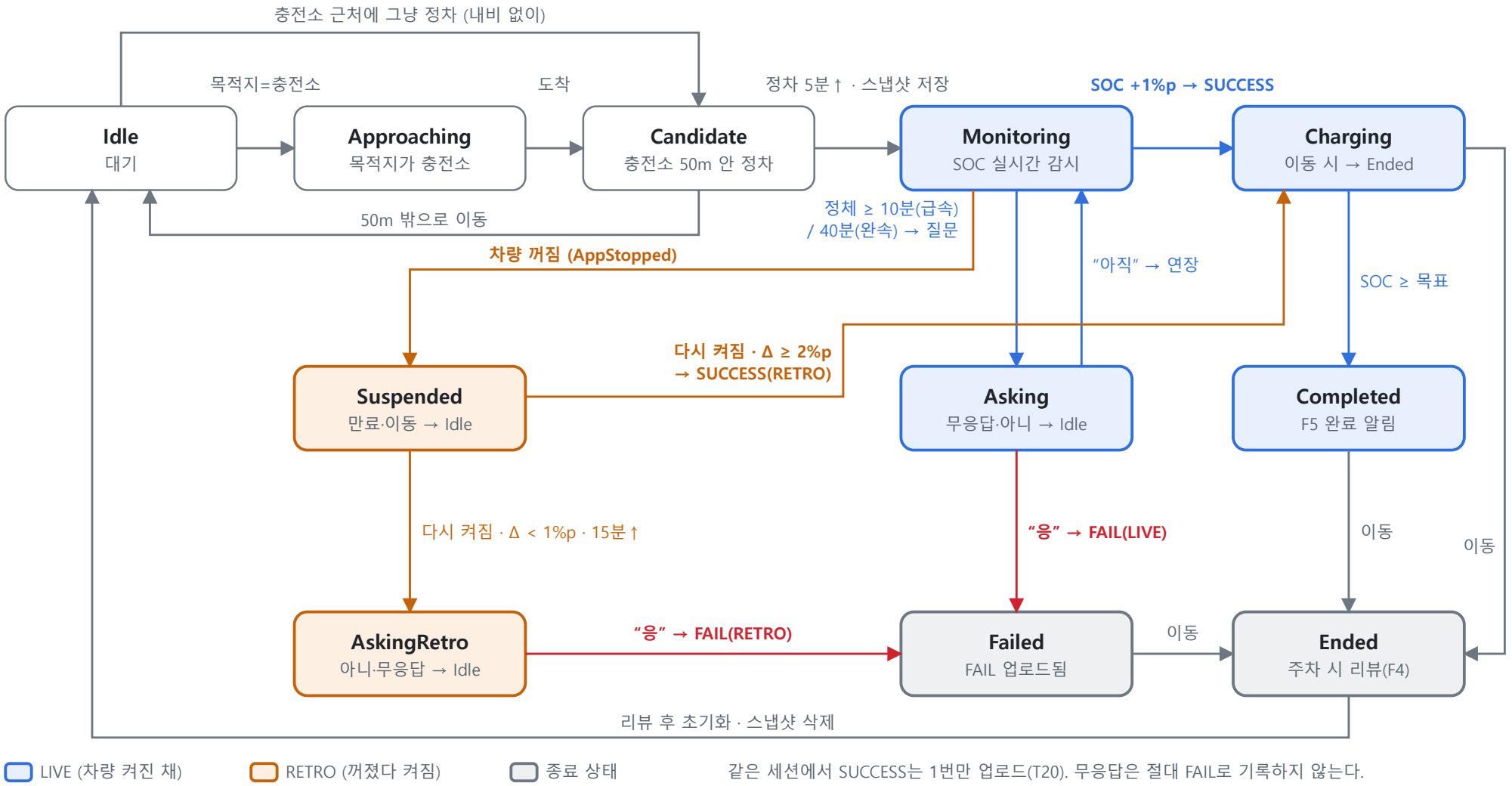


금지: feature → adapter, feature → Pleos SDK, adapter → feature. 'real'과 'demo'는 app 모듈의 Hilt 바인딩만 다르다.

3. 충전 세션 상태 머신 (ChargeSessionMachine)

F1 · F2 · F5 · 전이표 T1–T20

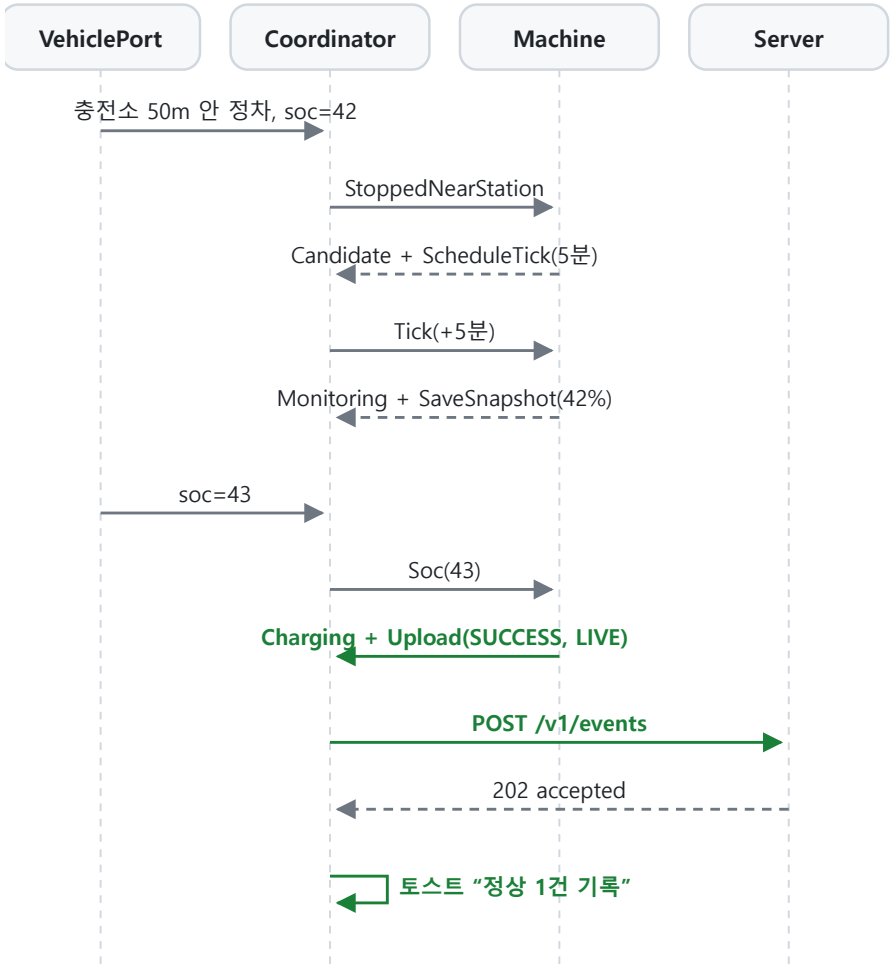
파란색은 차량이 켜진 채 실시간으로 보는 LIVE 경로, 주황색은 차량이 꺼졌다 켜졌을 때 비교하는 RETRO 경로다.



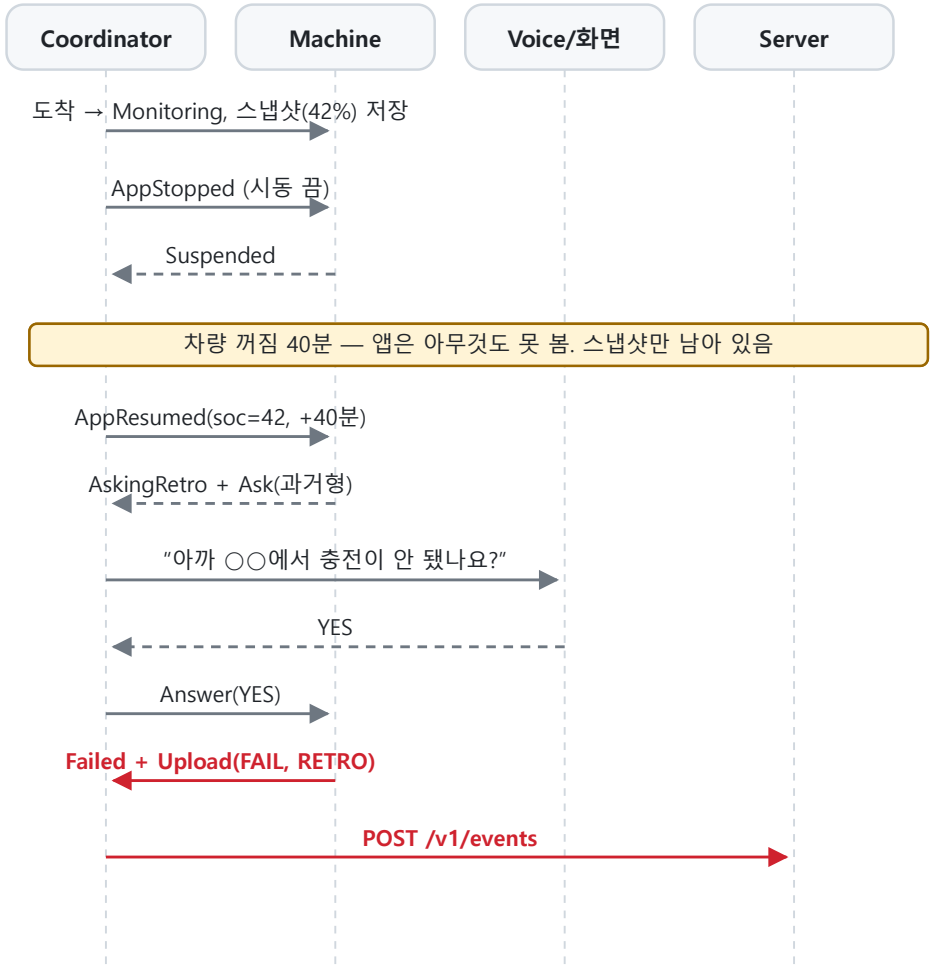
4. 핵심 흐름: LIVE 성공 · RETRO 실패

왼쪽: 충전 중 차 안에 있을 때. 오른쪽: 충전기를 꽂고 자리를 비웠다가 돌아왔는데 잔량이 그대로일 때.

LIVE 성공 — 차 안에서 충전을 기다릴 때



RETRO 실패 — 꽂고 자리를 비웠을 때

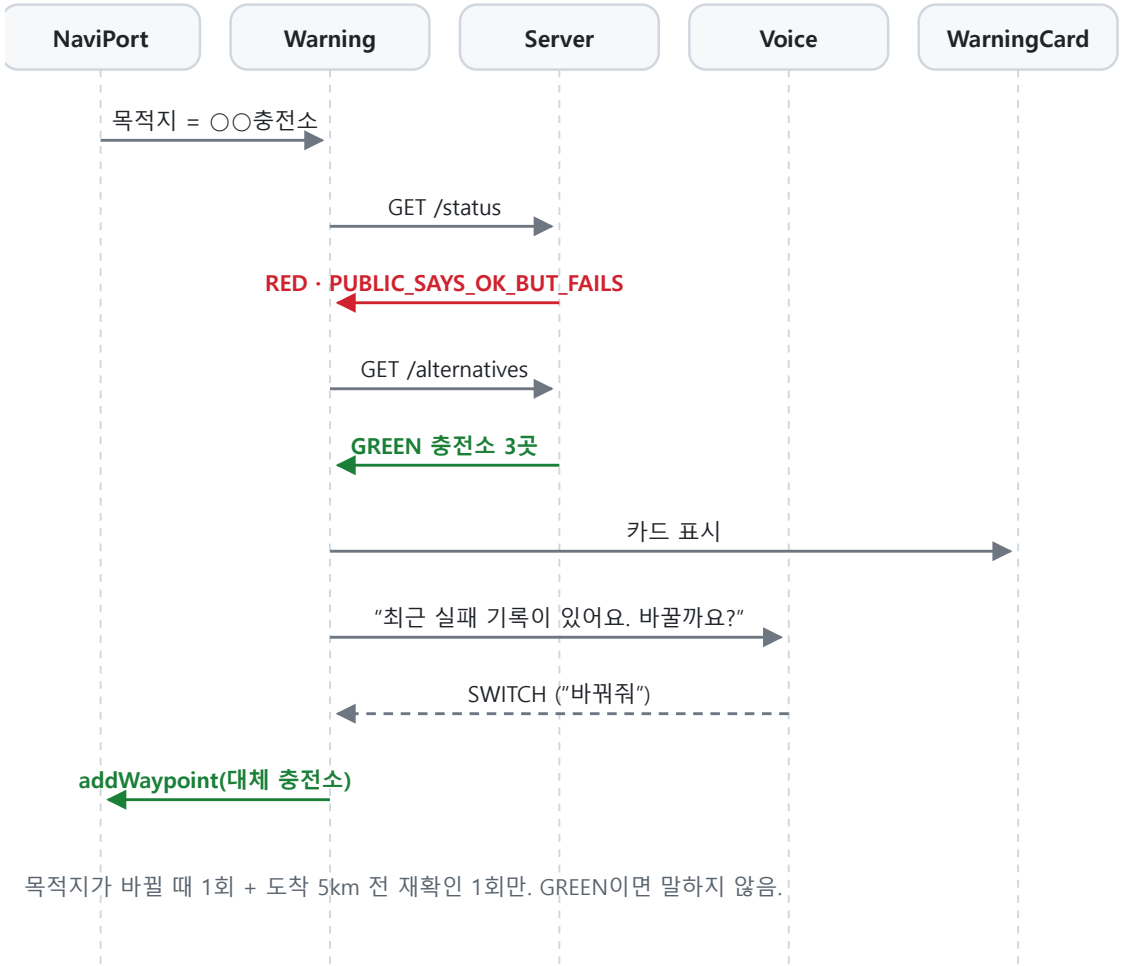


5. 도착 전 경고(F3)와 충전소 판별

ArrivalWarningCoordinator · StationResolver

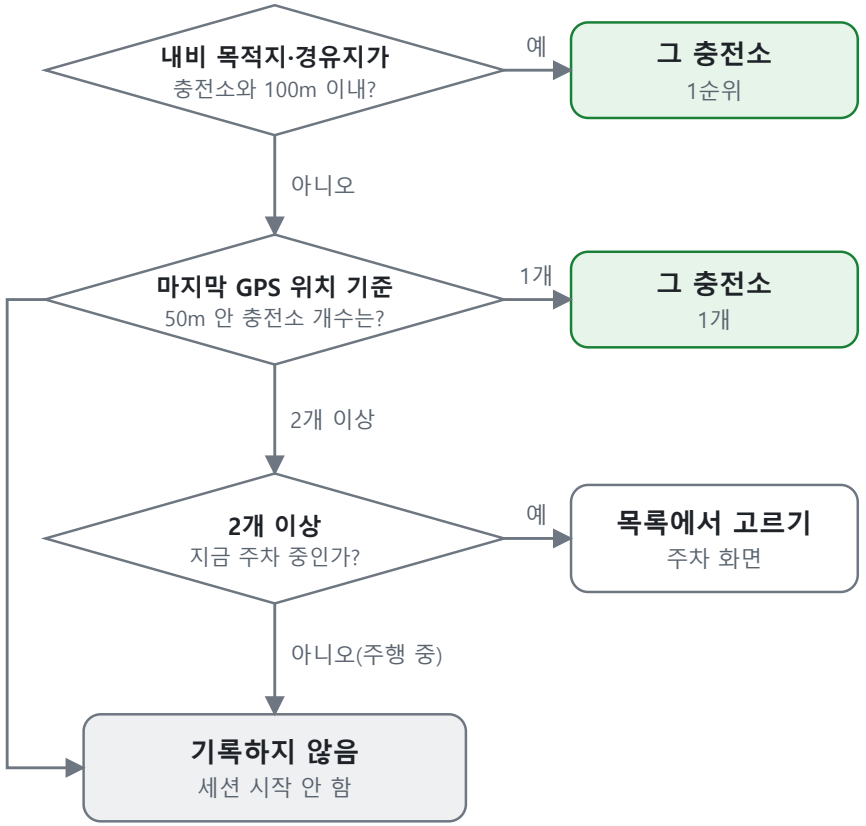
내비 목적지가 충전소면 서버 판정을 받아 경고한다. 어느 충전소인지는 내비 목적지를 먼저 믿고, 없으면 마지막 GPS 위치로 찾는다.

F3 도착 전 경고 (주행 중: 음성 + 버튼 2개)



목적지가 바뀔 때 1회 + 도착 5km 전 재확인 1회만. GREEN이면 말하지 않음.

StationResolver — 어느 충전소인가



지하 주차장은 GPS가 끊기므로 '마지막으로 잡힌 위치'를 쓴다.
충전소 목록은 nearby API로 받아 24시간 캐시.

6. 서버: API · 데이터 · 판정 규칙

FastAPI · PostgreSQL · compute_verdict

판정은 위에서부터 처음 맞는 규칙 하나만 적용한다. '공공 API는 사용 가능인데 실제 실패 2건'이 이 앱만 줄 수 있는 경고다.

REST API

POST	**/v1/events**	충전 결과 1건 (eventId로 중복 방지)
GET	**/v1/stations/{id}/status**	Verdict + 이유 + 공공 API 요약
GET	**/v1/stations/nearby**	반경 500m 충전소 (최대 30)
GET	**/v1/stations/{id}/alternatives**	GREEN 우선 대체 3곳 (5km)
POST	**/v1/reviews**	태그 평가

데이터 (좌표·이동 경로는 저장하지 않음)

station
- stat_id PK
- name · address
- lat · lon
- operator
- charger_type

charger
- (stat_id, chger_id) PK
- chger_type
- public_stat
- stat_upd_dt

charge_event
- event_id PK
- device_hash
- stat_id FK
- result · mode
- soc_start · soc_end
- minutes · occurred_at

review
- id PK
- device_hash · stat_id
- tags[] · created_at

charger · charge_event · review는 stat_id로 station을 참조

compute_verdict — 위에서부터 처음 맞는 규칙

1. **공공 API: 모든 충전기 운영중지·점검** — PUBLIC_OUT_OF_SERVICE → RED
   ↓ 아니면
2. **마지막 성공 뒤 실패 ≥ 2대 & 공공 API는 사용 가능** — PUBLIC_SAYS_OK_BUT_FAILS → RED
   ↓ 아니면
3. **마지막 성공 뒤 실패 ≥ 2대** — MULTI_FAIL → RED
   ↓ 아니면
4. **마지막 성공 뒤 실패 1대** — FAIL_AFTER_SUCCESS → YELLOW
   ↓ 아니면
5. **3시간 안에 성공 기록** — RECENT_SUCCESS → GREEN
   ↓ 아니면
6. **그 외 (24시간 기록 없음 등)** — NO_RECENT_DATA → GRAY

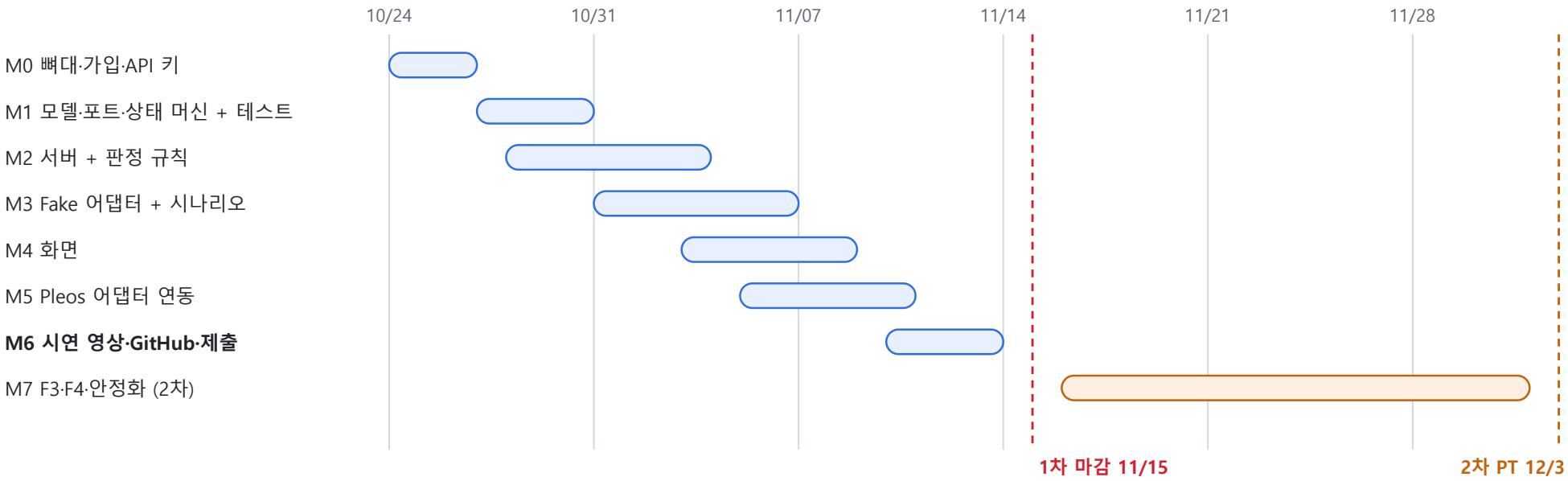
실패 수는 서로 다른 기기(device_hash) 기준. 같은 기기·충전소는 2시간에 1건만 인정.
한 기기의 실패가 하루 10건을 넘으면 그날 실패는 판정에서 제외.

2번이 이 앱만의 경고: "사용 가능으로 표시되지만 최근 실제로는 충전이 안 됐어요"

7. 구현 순서와 확인 안 된 것

11/15 1차 제출 · 12/3 2차 PT

상태 머신과 가짜 어댑터를 먼저 만들면, Pleos 연동 전에도 시연 영상을 찍을 수 있다.



확인 안 된 것	영향	대안 (설계에 반영됨)
시동이 꺼진 동안 Pleos 앱 실행 여부	LIVE 판정 범위	RETRO 경로(도착·출발 SOC 비교)
EvBattery 메서드 시그니처·갱신 빈도, 표준 EV_CHARGE_STATE 읽기 가능 여부	어댑터·감지 속도	로그인 후 API Reference 확인, 가능하면 chargeState() 추가
에뮬레이터에서 Android TTS/STT/위치 동작	음성·위치	화면 버튼 병행, NaviHelper 위치 이벤트
공공 API stat 코드 의미, getChargerInfo 필드	판정	PUBLIC_STAT_MAP 한 곳만 수정
차량 앱의 외부 서버 통신 심사 제약	서버 구조	10/28 온라인 설명회에서 질문